SLHE Game Programming Class

Engine & Platform Guide

VIDEO GAME CREATION

Please read before Lesson 1 - Thursday, August 13, 2026

Why This Guide

This class lets you choose your own game engine - there is no single required platform. Every lesson's core lecture is taught live in one engine, but you'll always get working example code and instructions for every engine we support, so you can follow along and build in whichever tool fits your device, your interests, and your goals. You're also welcome to switch engines partway through the year - that's one reason we keep parallel examples for everything.

This guide exists so you and your student can choose - and install - a platform before the first class. Installing large engines (especially Unity and Unreal) over shared classroom wifi with a room full of students is slow; please install at home ahead of time wherever possible. Lesson 1's lab time is reserved for installation help and troubleshooting, but a head start makes that time much more useful.

Quick Comparison

Engine	Coding Style	Cost	Min. RAM	Install	Best Fit
Scratch	Block	Free	~2 GB	None (browser) or tiny app	Absolute beginners, oldest laptops
Godot	Script (GDScript/C#)	Free	4 GB	~150 MB, no install	Best all-around, low-resource main engine
Unity	Script (C#) + Visual Scripting	Free (Personal)	8-16 GB	Multi-GB, Hub required	Industry-standard track, Unity Learn
GameMaker	Hybrid (GML / GML Visual)	Free tier	2-4 GB	Small installer	2D games, light hardware, visual to code bridge
GDevelop	Visual (event-based) + JS	Free	~2 GB	None (browser) or small app	Web fallback, weakest laptops
TIC-80	Script (Lua + others)	Free	~1 GB	Single tiny executable	Retro fantasy console, ultra-light
001 Game Creator	Hybrid (Visual + optional text)	Free trial; Edu from $30/seat/yr	512 MB	Small installer - Windows only	Lightest option here, classroom-ready - Windows-only
Unreal Engine	Blueprints + C++	Free	32 GB rec.	Tens of GB	Optional/advanced track, capable machines only

Which Engine Fits My Laptop?

As a Bring-Your-Own-Device class, hardware varies a lot. Use this as a starting filter - you can always try more than one.

Your Laptop	Recommended Engines
Older / 4-8 GB RAM, no dedicated graphics card	Scratch, TIC-80, GDevelop, GameMaker - all run comfortably
Mid-range / 8-16 GB RAM	Godot, GameMaker, GDevelop (desktop) - add Unity if closer to 16 GB
Newer / 16 GB+ RAM with a dedicated GPU	Unity comfortably; Unreal Engine becomes realistic (32 GB is Epic's recommended spec)
Any Windows laptop, even a very old one - but Windows only	001 Game Creator (512 MB RAM minimum) - the lightest option here, but there is no Mac or Linux editor

Engine Profiles

Each profile below covers coding style, cost, hardware needs, honest pros/cons, and step-by-step install instructions.

1. Scratch

Block coding - the on-ramp. Free, MIT-developed, runs on anything.

- **Coding style:** Pure block/drag-and-drop coding. No text code option.
- **Cost / licensing:** Completely free, no account required to use the offline editor.
- **System requirements:** Any laptop from the last 15 years. Runs in a browser, or as a tiny offline desktop app.
- **Install difficulty:** Trivial - use it directly at scratch.mit.edu, or install the free offline editor (no internet needed in class).

Pros

- Zero hardware risk - works on the oldest device in the room.
- Fastest path to a finished, playable game for a first-time coder.
- Huge library of kid-safe example projects to remix.

Cons

- Block-only - no path to real text-based code inside Scratch itself.
- Can feel young for older high schoolers if used for too long.

Download & Install

1. Go to scratch.mit.edu/download and choose your OS (Windows/macOS/Linux), OR just use it in a browser at scratch.mit.edu - no login needed to experiment.
2. Offline app: run the installer, no additional setup.
3. Optional: create a free Scratch account to save projects to the cloud and share with classmates.

Learn more: scratch.mit.edu

2. Godot Engine

Our recommended default - free, light, and a real scripting language.

- **Coding style:** Traditional text scripting in GDScript (Python-like, beginner-friendly) or C#.
- **Cost / licensing:** 100% free forever. Open source (MIT license). No account, no revenue thresholds, nothing to buy.
- **System requirements:** Comfortable minimum: dual-core CPU, 4 GB RAM, any GPU supporting OpenGL 3.3 (Godot's Compatibility rendering mode covers older integrated graphics). Recommended: 8 GB RAM.
- **Install difficulty:** Easiest real-engine install on this list - a single ~150 MB file, no installer, no account.

Pros

- Best balance of real capability and low hardware demand.
- GDScript is one of the gentlest true coding languages to learn.
- No licensing strings attached, ever - good for families wary of subscriptions.

Cons

- Smaller tutorial/certification ecosystem than Unity.
- Less name-recognition on a resume than Unity or Unreal (though this is changing fast).

Download & Install

4. Go to godotengine.org/download and select your OS.
5. Download the Standard build (skip .NET/C# build unless you plan to code in C#).
6. Unzip and run the single executable - nothing to install.
7. If your laptop is older, go to Editor Settings > Rendering and switch to Compatibility renderer if the default is slow.

Learn more: godotengine.org

3. Unity

The industry-standard track - free to learn, with a full certification path.

- **Coding style:** Traditional text scripting in C#. A built-in Visual Scripting package (node graphs) is also available - see the Hybrid Engines section below.
- **Cost / licensing:** Unity Personal is free with no seat limit as of 2026 (Personal is capped by company revenue, which does not apply to student hobby projects).
- **System requirements:** 8 GB RAM is Unity's stated floor; 16 GB is a realistic comfort level for the Editor plus a class project. Any modern integrated GPU works for 2D/simple 3D.
- **Install difficulty:** Moderate - install Unity Hub first, then a specific Editor version (multi-GB download) plus Visual Studio Code for C#. Budget real time/bandwidth for this on install day.

Pros

- Massive learning ecosystem: Unity Learn is free, with guided Pathways (Essentials, Junior Programmer, Creative Core, Game Development).
- A genuine path to industry certification (Pearson VUE exams, paid, with free prep material).
- C# is a widely-used, resume-relevant language.

Cons

- Heaviest install/update footprint of the main engines besides Unreal.
- C# has a steeper first-week learning curve than GDScript or block coding - better suited to older or more experienced students.

Download & Install

8. Go to unity.com/download and install Unity Hub.

9. In Unity Hub, install the latest LTS Editor version (this is the big download - several GB; do this on home wifi before class if possible).
10. Install Visual Studio Code when prompted (Unity's recommended free code editor).
11. Create a free Unity ID account to activate Unity Personal.
12. Create a new 2D or 3D project from Unity Hub to confirm everything works.

Learn more: learn.unity.com

Older laptop, GPU errors on install day? Some older/business laptops only have partial DirectX 11 support and will throw GPU fence or shader errors under Unity 6.x. See the companion troubleshooting document "Older_GPU_DirectX_Troubleshooting.docx" (in this same folder) for the two fixes - an in-editor settings change, or downgrading to Unity 2023 LTS.

4. GameMaker

A light 2D engine with a genuine visual-to-code bridge.

- **Coding style:** Hybrid: GML (a C-like scripting language) or GML Visual (drag-and-drop blocks that generate real GML code live - a great stepping stone). See the Hybrid Engines section below for how the two connect.
- **Cost / licensing:** Free tier (non-commercial) has no seat limits and covers everything needed for class. A one-time $99.99 Professional license is only needed if a student wants to publish/sell a finished game outside class.
- **System requirements:** Very light: 2 GB RAM minimum (8 GB comfortable), any OpenGL 4-class integrated graphics, ~3 GB disk space.
- **Install difficulty:** Simple installer, everything bundled - no separate compiler to configure.

Pros

- Because GML Visual shows the real code being generated as you drag blocks, it's a uniquely good bridge from visual to text coding.
- Comfortably runs on aging BYOD hardware.
- 2D-focused, which matches most first student game projects well.

Cons

- 2D only - no 3D path if a student wants to go there later.
- Publishing a finished game outside of class requires the paid Professional license - worth flagging to families in advance.

Download & Install

13. Go to gamemaker.io/download.
14. Create a free account and choose the Free (non-commercial) plan.
15. Run the installer for your OS.
16. Sign in on first launch to activate the free license.

Learn more: gamemaker.io

5. GDevelop - the web/no-install fallback

Our recommended fallback for the weakest laptops or a bad install day.

- **Coding style:** Visual, event-based programming (conditions to actions) by default, with inline JavaScript code blocks available for advanced logic - see the Hybrid Engines section below.
- **Cost / licensing:** Completely free and open source. No feature caps, no account required for the core experience.
- **System requirements:** Runs entirely in a modern browser (Chrome/Firefox) - effectively no hardware requirement. A lightweight desktop app is also available for offline use.
- **Install difficulty:** Easiest possible - open a browser tab, or install a small desktop app if you want to work offline.

Pros

- Zero-install option - the true fallback for a laptop that won't cooperate that day.
- Not meaningfully behind Godot for the scope of 2D games this class will build in a semester.
- Free with no feature ceiling, unlike some competing browser tools.

Cons

- Smaller tutorial community and less industry name-recognition than Unity/Godot.
- Not the tool of choice for high-end 3D work.

Download & Install

17. Go to gdevelop.io and click Create a game to use it instantly in-browser - no install needed.
18. For offline use: download the desktop app for your OS from the same site and run the installer.

Learn more: gdevelop.io

6. TIC-80 - the retro option

A tiny fantasy console for learning fundamentals without a huge project scope.

- **Coding style:** Traditional text scripting - choose from Lua, JavaScript, Python, Ruby, Wren, Fennel, Squirrel, or Moonscript, all in the same tool.
- **Cost / licensing:** Free and open source.
- **System requirements:** Trivial - runs well even on a Raspberry Pi. Any laptop handles this easily.
- **Install difficulty:** Single small executable, no dependencies, no account.

Pros

- Built-in constraints (small screen, 16-color palette) force small, genuinely finishable projects - great for building confidence.
- Language flexibility means a stronger student can pivot to Python or JavaScript within the same tool.

- Works identically well on the oldest laptop in the room.

Cons

- Smaller built-in tutorial/community base than its paid cousin PICO-8 (a similar concept but costs $15 and has more third-party tutorials - not required for this class).
- Intentionally limited scope - not a path to a modern 3D game.

Download & Install

19. Go to tic80.com and download the free build for your OS.
20. Unzip and run the executable - no installer needed.
21. Type 'new lua' (or the language of your choice) at the built-in console to start a project.

Learn more: tic80.com

7. 001 Game Creator - the classroom-built option

A dedicated no-coding engine, built by SoftWeir with classrooms specifically in mind. Windows only.

- **Coding style:** Hybrid: graphical, node-based scripting by default (with plain-language tooltips instead of programming jargon), plus an optional text-code editor for the same underlying logic - see the Hybrid Engines section below for an important caveat about switching.
- **Cost / licensing:** Standalone one-time purchase, normally $79.99 (frequently on sale around $20) - no royalties or income limits after that. Separately, a purpose-built Educational tier exists: a free Basic plan (watermarked output, 10 maps/project, exporting disabled) or paid per-workstation/per-year seats - Essentials ($30/workstation/year, unlocks Windows export) and Premium ($50/workstation/year, adds mobile/web/Steam export plus a bundled E-Book curriculum with exercises, assignments, and quizzes).
- **System requirements:** By far the lightest engine on this list: minimum 1.0 GHz CPU, 512 MB RAM, 1 GB disk, OpenGL 2.1, Windows 7. Recommended: 2.0 GHz CPU, 1 GB RAM, OpenGL 3.0, Windows 11.
- **Install difficulty:** Simple installer. Important: the development editor is Windows-only - there is no Mac or Linux version. (Exported games can still target Windows, Steam, HTML5, Android, and iOS.)

Pros

- Lightest system requirements of any engine here - runs on essentially any Windows laptop, no matter how old.
- The Educational tier is genuinely built for a classroom: content is filtered (no blood/gore, weapons, or profanity), seats are licensed per workstation, and an optional E-Book bundle ships with exercises, assignments, and quizzes already written.
- Graphical scripting with plain-language tooltips can be a good fit for students intimidated by traditional code syntax, while still supporting text scripting for those who want it.

Cons

- Windows only for development - students on a Mac or Chromebook cannot install the editor at all, a real limitation for a BYOD class.

- The free Basic tier cannot export or build a shareable game; a paid Essentials seat ($30/workstation/year) is the practical minimum for a student who wants a build to share at the CoOp Expo.
- Much smaller community, tutorial base, and industry footprint than Unity, Godot, or GameMaker.

Download & Install

22. Confirm you're on Windows 7 or later - there is no Mac/Linux editor for this engine.
23. Go to 001gamecreator.com/downloads and try the free 7-day trial (full features except exporting/saving after the trial ends), or go to 001gamecreator.com/educational to inquire about a classroom Educational license.
24. Run the installer.
25. To get an exportable, shareable build for the Semester 2 capstone, a paid Standalone license or an Educational Essentials/Premium seat is required - the free tiers do not support exporting a finished build.

Learn more: 001gamecreator.com

8. Unreal Engine - optional advanced track

The most powerful engine here, and the heaviest. Available to students with capable hardware.

- **Coding style:** True hybrid: Blueprints (visual scripting, can build a complete game with zero text code) or C++ - see the Hybrid Engines section below for how the two work together.
- **Cost / licensing:** Free to download and use. 0% royalty until $1,000,000 in lifetime revenue - irrelevant for classroom use.
- **System requirements:** Epic's own recommended spec is around 32 GB RAM and a GPU with 8 GB+ VRAM; an SSD is effectively required. This is beyond most BYOD laptops.
- **Install difficulty:** Heaviest of all options - tens of GB, often paired with a full Visual Studio install for C++ work.

Pros

- Blueprints offer a real visual-scripting path to a fully playable game, not just a toy.
- Highest ceiling for students who want to go further into professional 3D production.

Cons

- Not realistic for most BYOD laptops, especially older ones - this is confirmed by Epic's own recommended specs.
- Longest install time - plan for a home download, not a shared-wifi lab download.

Download & Install

26. Only attempt this if your laptop has 16 GB+ RAM and a dedicated graphics card.
27. Create a free Epic Games account at unrealengine.com.
28. Install the Epic Games Launcher, then install Unreal Engine from within it (this is a large download - do it at home on good wifi, not during class).

29. For C++ work, also install Visual Studio Community (free) with the Game development with C++ workload.

Learn more: unrealengine.com

Hybrid Engines: Switching Between Styles

Three distinct coding styles run through this whole class: Block (Scratch), Visual/node-based (GDevelop, GameMaker's GML Visual, Unity's Visual Scripting, 001 Game Creator, Unreal's Blueprints), and Scripting/text-based (Godot, GameMaker's GML, Unity's C#, TIC-80). Five of our engines are hybrids that offer two of these styles inside the very same project. That makes them worth a closer look on their own: a hybrid engine lets a student start in whichever style is comfortable and grow toward the other without switching tools, and it's a great case study in the difference between the styles, since you can watch the same logic represented two different ways. The five hybrid engines don't all work the same way, though - some let you convert one direction only, some don't convert at all but let both styles call into each other, and one shouldn't really be converted in class at all. The breakdown below is what to actually expect from each.

Unity - Visual Scripting <-> C#

- **Two styles:** Visual Scripting (node graphs) and C# (text code).
- **How they interact:** They coexist in the same project rather than converting into each other - different GameObjects can use either system, and a Visual Scripting graph can call directly into your own C# classes.
- **What you gain switching:** A student can start building real logic with nodes before learning C# syntax, then call into hand-written C# functions as their skills grow.
- **What's limited:** There is no official "convert this graph to C#" button - a graph stays a graph. Graphs reference C# code by name/reflection, so renaming a C# method can silently break a graph node that used it.

Think of Unity's two styles as two toolkits that can talk to each other, not two views of the same logic.

GameMaker - GML Visual <-> GML

- **Two styles:** GML Visual (drag-and-drop icon blocks) and GML (C-like text code).
- **How they interact:** Right-click any event's action list and choose "Convert To GML Code" to turn that one event's blocks into real GML text - conversion happens per event, not for the whole project at once.
- **What you gain switching:** Students see the actual code their blocks were generating the whole time - one of the best visual-to-text "aha" moments in the whole curriculum.
- **What's limited:** The conversion is one-way. GameMaker warns before converting that the generated text cannot be turned back into blocks - once converted, that event is GML from then on.

Use "Convert to GML Code" deliberately, and on a simple event or a copy first - a class demo shouldn't accidentally destroy someone's visual work.

001 Game Creator - Script Editor <-> Textual Scripting

- **Two styles:** The node-based Script Editor (default) and Textual Scripting (a BASIC-like text mode).

- **How they interact:** A script's mode can be switched from Graphical to Textual.
- **What you gain switching:** A curious student can see what plain, BASIC-style code looks like under the hood.
- **What's limited:** Per 001 Game Creator's own wiki, this conversion is one-way and destructive - switching back to Graphical loses the original events - and the wiki states plainly that Textual Scripting "offers no advantages over visual scripting" and is considered legacy.

Treat 001 Game Creator as visual-only in this class. Textual Scripting is worth knowing about, but not worth building on.

Unreal Engine - Blueprints <-> C++

- **Two styles:** Blueprints (visual node graphs) and C++ (text code).
- **How they interact:** This is the one true two-way bridge on our list. C++ functions and classes can be exposed directly to Blueprints (using UFUNCTION/UCLASS), Blueprint graphs can call them, and a Blueprint class can subclass a C++ parent class - both styles run in the same project and call into each other freely.
- **What you gain switching:** This mirrors how professional Unreal teams actually work: designers build and iterate in Blueprints while programmers expose gameplay systems from C++.
- **What's limited:** There's no automatic "Blueprint to C++" converter available to us - an older feature called Blueprint Nativization did this, but Epic has deprecated it. Converting a Blueprint to C++ today means a programmer manually rewriting it by hand.

Unreal is the one engine where "switching styles" really means designers and programmers collaborating across styles, not one style converting into the other. (Forward-looking note: Epic has announced Blueprints and the classic Actor system are slated for eventual replacement by a new language called Verse in UE6 - but our target version, UE5, keeps Blueprints fully supported, so nothing changes for this class.)

GDevelop - Event Sheets <-> JavaScript

- **Two styles:** Visual event sheets (conditions -> actions) and inline JavaScript code blocks.
- **How they interact:** The two are freely mixed - a JavaScript Code event can sit right inside a normal event sheet alongside ordinary visual conditions and actions.
- **What you gain switching:** A JavaScript Code event is a trapdoor for anything the visual event system can't express - genuinely useful once a student outgrows the built-in actions.
- **What's limited:** A JavaScript Code event doesn't automatically see every object in your scene - you have to explicitly choose which objects to "pass in" to the code block first.

GDevelop's mixing is the most flexible of the five - closer to Unreal's two-way bridge than to GameMaker's one-way door.

The 30-Lesson Master Schedule calls out two "Style Spotlight" moments where this happens live in lecture: GameMaker reappears in Lesson 9 in GML text mode after being demoed in GML Visual mode back in Lesson 5, and Unity reappears in Lesson 12 in C# mode after being demoed in Visual Scripting mode in Lesson 7. Both are a chance to watch the same kind of problem solved in the other style, back to back, a few weeks apart.

A Note on the Name Game Creator

This class supports 001 Game Creator (001gamecreator.com, by SoftWeir) - see its full profile above. If you search around and see a different tool called Game Creator 2, that's an unrelated paid visual-scripting add-on built on top of Unity, not the engine we mean. We mention it here only so families researching options aren't thrown off by the similar name.

Frequently Asked Questions

Do I have to buy anything?

No. Every engine in the core lineup (Scratch, Godot, Unity, GameMaker, GDevelop, TIC-80) is free for classroom use. Unreal Engine is also free, but requires stronger hardware. 001 Game Creator has a free trial and a free Educational Basic tier, though exporting a finished game requires a paid tier (Standalone purchase, or an Educational Essentials/Premium seat).

What if my laptop is really old or slow?

Start with Scratch, GDevelop (in a browser - no install at all), TIC-80, or (if you're on Windows) 001 Game Creator. All run comfortably on hardware from a decade or more ago, and none will be a wasted investment of time.

Can I change engines partway through the year?

Yes. Because every lesson includes examples across all supported engines, switching is always an option - you won't be starting from zero.

Which engine should I pick if I just want the safest, most flexible choice?

Godot. It's free with no strings attached, runs on almost anything, and teaches a real scripting language without Unity's heavier install and steeper first weeks.

Which engine has the strongest path to an industry certification?

Unity, via the free Unity Learn Pathways and the paid Pearson VUE certification exams.

My laptop is a Mac or Chromebook - can I use 001 Game Creator?

No - 001 Game Creator's development editor only runs on Windows. Mac and Chromebook students should choose from Scratch, Godot, Unity, GameMaker, GDevelop, or TIC-80 instead; during any week 001 Game Creator is featured in lecture, you'll follow the same lesson using your own engine's parallel example.

What are Block, Visual, and Scripting, and why do they keep getting mentioned?

They're the three broad styles of writing game logic used across this class: Block (drag-and-snap puzzle pieces, like Scratch), Visual/node-based (connecting events, conditions, and actions without typing syntax, like GDevelop or Blueprints), and Scripting (typed text code in a real programming language, like GDScript or C#). Every lesson's featured engine rotates through all three roughly evenly across the year, so you'll get real exposure to each - see the Hybrid Engines section above and the 30-Lesson Master Schedule's Coding Style column.